WASL Arena

One family. Three games. One fair AI referee.

Family AI Game Day · Technical report · 6th Annual COE STEM Programming with AI Competition, Abu Dhabi University

Team 204 · Team code AYUCK · Track: Cycle 3 (grades 9-12)

May Ahmed Mourad (G12) · Shamsa Faris Almazrouei (G12) · Fatima Mohamed Almazrouei (G11)

1 Executive summary, problem and objectives

WASL Arena is a working web application that lets a family play three games, **football plus two original games**, while an AI referee watches the recorded video and listens to the audio, finds what actually happened, applies the written rules and keeps score. A human host runs the session: registering players and teams, running rounds and timers, and reviewing any call the AI is unsure about. The defining idea is **AI Referee Replay**: every score change is linked to evidence you can replay: the source video, timestamps, annotated frames, the rule applied, the points awarded and a verification status. If the AI cannot see it, it does not score it: the event is marked **NEEDS_REVIEW** and waits for the host.

The problem

Family games are decided by arguments. “Was that a goal?” “Did you say it before you showed the card?” An AI judge could settle these fairly, but only if families can trust it. Black-box scoring fails that test, and so does an AI that guesses. The challenge asks for an app whose AI **observes real play through video and audio, applies the rules and keeps score**, with a human host in charge and a demo that shows the AI judging working on real gameplay.

Objectives

- Deliver **exactly three games**: Football (required), **Falaj Flow** (team) and **Souq Signal** (individual), covering team and individual play and avoiding the example games excluded by the brief.
- Judge **real video/audio** from an upload or direct camera/microphone recording using computer vision and speech transcription, not simulated inputs.
- Make every judgment **explainable, verifiable and tamper-evident**, with uncertain detections routed to the host.
- Support the host journey end to end: choose a game, players and teams, rules and readiness, rounds and turns, AI judging, results with reasons and MVP, play another game.

How the brief is met

Official requirement	WASL Arena implementation	Status
Football + 2 original games (Cycle 3)	Football AI · Falaj Flow · Souq Signal	Built
Team and individual competition	Football, Falaj = teams · Souq Signal = individual	Built
Upload video with audio or record from camera/mic	File upload and MediaRecorder capture in the Live Arena	Built
AI follows players/ball; goals, possession, MVP	Colour tracking, goal zones, observed-possession MVP	Built; synthetic-tested
Host sets up, runs rounds/turns/timers, shows reasons	Setup wizard, turn order, timer, Replay with reasons	Built
AI report verified by code or hash	SHA-256 + HMAC seal; stand-alone Python judge; import rejects edits	Built; tested
Demo with real gameplay	Pipeline proven on synthetic fixtures; **real clips pending recording**	Pending real footage

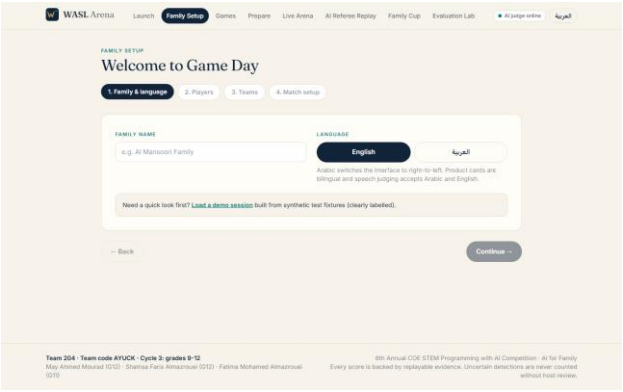
2 System concept, the three games and the user journey

WASL (وصل) means "connection": the app connects physical family play to digital judging. The same flow serves all games: **1 Choose a game → 2 Players and teams → 3 Rules and ready → 4 Round in play → 5 AI judge results → 6 Final results**, exactly the host journey in the brief.

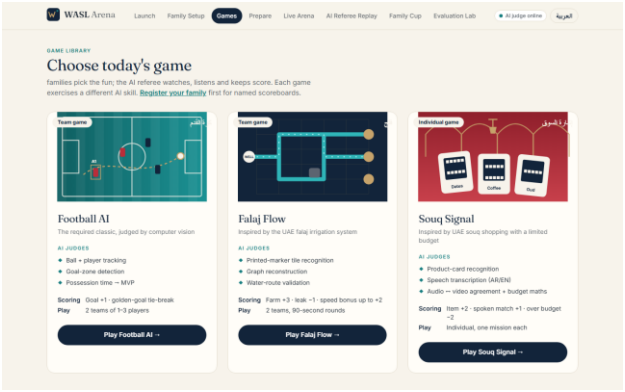
Game	Mode	What families do	What the AI does	Scoring
Football AI	Team	Mini-match with a bright ball, two goals, contrasting shirts, fixed camera	Tracks ball and players, detects goals, measures possession time per team and player	Goal +1; tie → golden goal; MVP = longest possession
Falaj Flow (فلج)	Team	Build a water-channel network on a printed 5×5 board with printed tiles in 90 s (inspired by the UAE falaj irrigation system)	Reads every tile and rotation, rebuilds the network as a graph, traces water to three farms	Farm +3; leak −1; speed bonus ≤ +2; tie → shortest network, time
Souq Signal (إشارة السوق)	Individual	Shopping mission with a budget: pick product cards, show them and name them aloud (inspired by UAE souqs)	Recognises cards, transcribes Arabic/English speech, checks audio and video agree, does the budget maths	Item +2; spoken match +1; over budget −2; duplicates 0

Each game exercises a different AI capability (motion tracking; visual recognition and graph reasoning; multimodal vision plus speech), which keeps the collection varied and gives judges three distinct demonstrations of the same fairness principle.

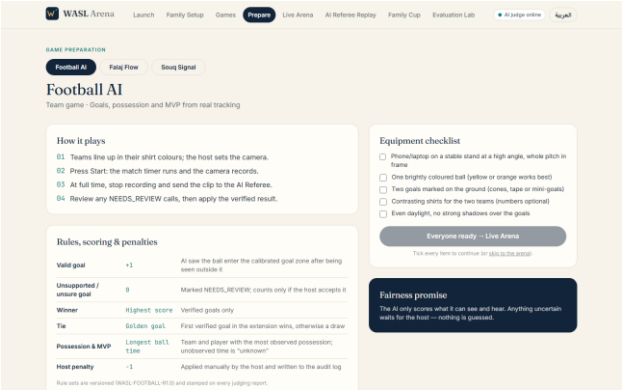
User journey (screens of the working prototype)



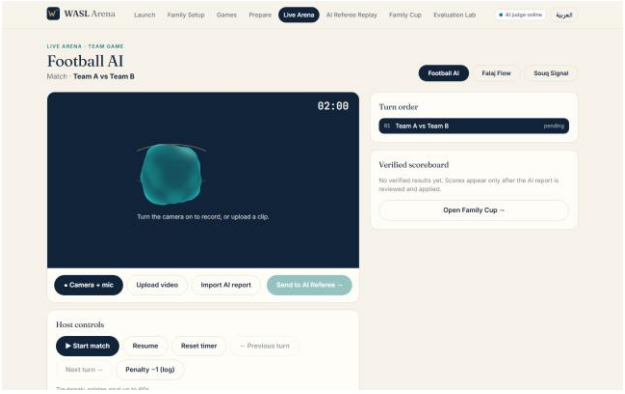
Family setup: players, teams, language, match rules



Game library: exactly three games



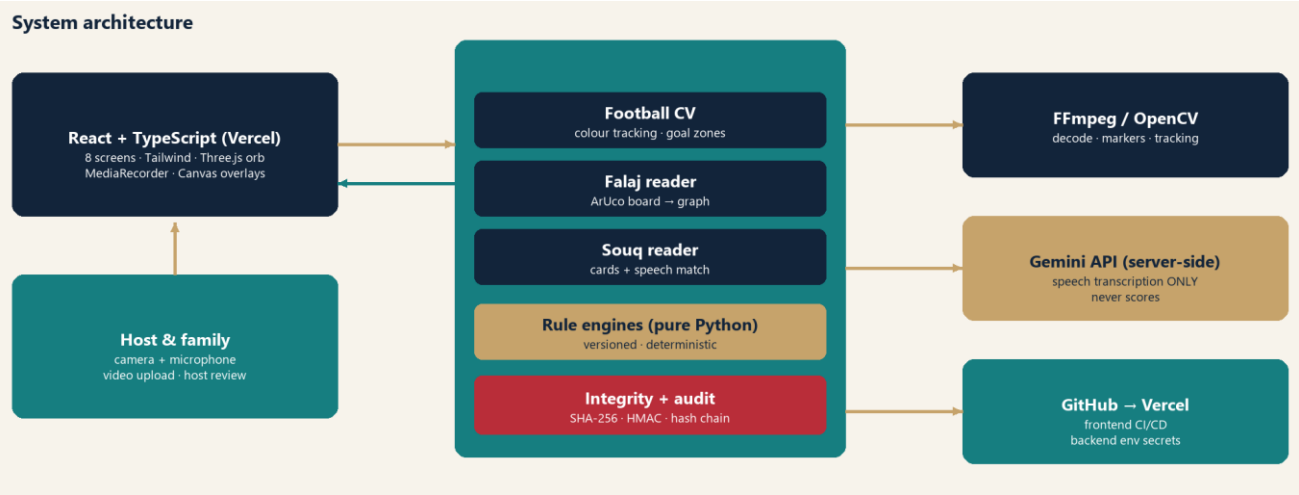
Preparation: rules, calibration, equipment, printables



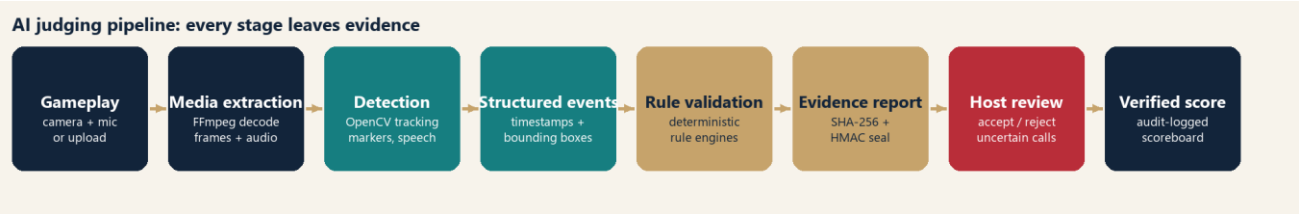
Live Arena: camera, timer, turns, verified scoreboard

Eight screens are implemented: **Launch, Family Setup, Game Library, Game Preparation, Live Arena, AI Referee Replay, Family Cup, Evaluation Lab**. The interface is bilingual (English/Arabic with right-to-left layout), and printable boards, tiles and product cards (with the exact markers the AI reads) are downloadable from the preparation screen.

3 System architecture, technology and AI pipeline



Layer	Technology	Role
Frontend	React 19, TypeScript, Vite, Tailwind CSS 4, React Router, Three.js	Eight responsive screens; MediaRecorder capture; Canvas overlays synchronised with video; original reactive 3D AI-Referee orb with CSS fallback
Backend	Python, FastAPI, Pydantic, Uvicorn	Upload, background judging jobs with real stage reporting, report store, host review, audit chain
Vision / media	OpenCV (HSV segmentation, contours, ArUco, homography), NumPy, FFmpeg	Frame sampling at 4–10 fps, object tracking, marker decoding, audio extraction
Speech	Gemini API (server-side key)	**Transcription only**: timed text segments; an energy gate drops segments with no real sound
Rules	Pure-Python, versioned rule engines	WASL-FOOTBALL-R1.0, WASL-FALAJ-R1.0, WASL-SOUQ-R1.0: the only scoring authority
Delivery	GitHub → Vercel (frontend); Render (backend)	Backend URL via VITE_API_URL; credentials only in server environment variables



Evidence integrity

Each job is hashed at upload (**SHA-256 of the original file**). The finished report carries the analysed-file hash, the rule version, event timestamps, bounding boxes, annotated evidence frames and validation checks, then is **sealed with a SHA-256 digest and HMAC signature**. Anyone holding the key can detect a single changed character; the app rejects tampered reports on import. Host decisions recompute the score through the same rule engine, are appended to a **hash-chained audit log**, and re-seal the report while preserving the original AI seal. The stand-alone judge (`python -m app.judge_cli`) produces the same sealed report outside the app, satisfying the brief's second connection option.

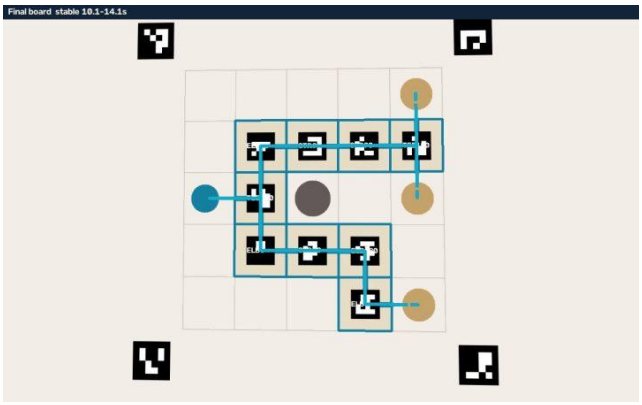
4 Football, Falaj Flow and Souq Signal: AI methodologies

Football AI: tracking, goals, possession and MVP

Frames are sampled at 10 fps and downscaled to 640 px. The **ball** is the best circular contour inside a calibrated HSV colour range (area and circularity gated, nearest to a velocity-predicted position). **Players** are team-colour blobs (morphological open/close, area filter) tracked by greedy nearest-neighbour association with 3-second re-association and a per-team player cap. A **goal** is the ball observed inside the calibrated goal polygon for ≥ 3 consecutive observed samples after being seen outside it, with a 3 s cooldown; a goal whose entry was not observed, or that lasts fewer samples, is NEEDS_REVIEW. A ball that heads for a goal and then disappears becomes a **POSSIBLE_GOAL** (never auto-scored). **Possession** credits the nearest player within 0.45 body-lengths for ≥ 3 samples; contested moments and unobserved time are credited to nobody. **MVP** is the player with the longest verified possession (as defined in the brief); because shirt numbers are not read, the host assigns colour tracks to registered players.



Football goal evidence frame (synthetic fixture)



Falaj: board read from video, water path (teal)

Falaj Flow: board recognition, graph reconstruction and validation

Four ArUco corner markers define a homography from the camera image to the 5×5 board. Each tile carries an ArUco marker whose **id gives the tile type** (straight, elbow, tee, cross) and whose **in-plane rotation gives the tile rotation** in 90° steps. A board arrangement counts only when it is **stable for 1 s**, so hands moving over tiles are ignored. The arrangement becomes a graph; breadth-first search from the well follows only open sides whose neighbour has a matching open side. An open end that meets the board edge, a rock, an empty cell or a closed side is a **leak**. Output: farms reached (+3 each), leaks (−1), time bonus by completion time, network size for the tie-break, and a digital twin that animates water only along rule-validated edges.

Souq Signal: multimodal verification

Each product card carries a unique marker (two copies per product); sightings are merged into **presentation intervals** (≥ 0.8 s, gaps ≤ 2 s). Speech is transcribed server-side, then matched to bilingual product aliases (Arabic normalisation, ال/و prefixes) and timestamped inside the segment. The rule engine then checks: requested item shown (+2), spoken within ±5 s (+1), **audio ≠ video mismatch → host review**, duplicate card (0), and distinct card prices against the budget (−2 once). With no transcript, spoken-match points stay NEEDS_REVIEW. Gemini never decides points: it provides text; deterministic code scores.

5 Testing, results, validation and limitations

All results below were produced by running the code (generated 2026-10-29). **46 of 46 automated tests passed, 0 failed.** Tests cover the three rule engines, the full CV-plus-rules pipelines, report sealing, tamper detection, the upload → job → report → host-review API flow and the audit chain.

Fixture (synthetic render)	Ground truth	Pipeline output	Result
football_goal	goals_A=1, goals_B=0, possession_leader=A	goals_A=1, goals_B=0, verified_goal_events=1, possible_goal=None, possession_leader=A	PASS
football_nogoal	goals_A=0, goals_B=0	goals_A=0, goals_B=0, verified_goal_events=0, possible_goal=None, possession_leader=A	PASS
football_uncertain	goals_A=0, possible_goal=NEEDS_REVIEW	goals_A=0, goals_B=0, verified_goal_events=0, possible_goal=NEEDS_REVIEW, possession_leader=A	PASS
falaj_valid	destinations=3, leaks=0, time_bonus=2, total=11	destinations=3, leaks=0, time_bonus=2, total=11	PASS
falaj_partial	destinations=2, leaks=1, time_bonus=0, total=5	destinations=2, leaks=1, time_bonus=0, total=5	PASS
falaj_rock	destinations=3, leaks=1, time_bonus=2, total=10	destinations=3, leaks=1, time_bonus=2, total=10	PASS
souq_correct	total=9, spend=55	total=9, spend=55, mismatch=False, duplicate=False, budget_exceeded=False	PASS
souq_mismatch	mismatch=True, total=2	total=2, spend=12, mismatch=True, duplicate=False, budget_exceeded=False	PASS
souq_budget_dup	duplicate=True, budget_exceeded=True, total=7	total=7, spend=55, mismatch=False, duplicate=True, budget_exceeded=True	PASS

Test module	Covers	Passed
falaj_rules	complete route, partial, blocked, invalid connection, scoring, tie-break	9/9
football_rules	valid goal, no goal, uncertain, score update, tie-break, MVP, penalties	8/8
integrity_api	seal/tamper, API flow, host review recompute, import rejection, audit chain	7/7
pipelines	end-to-end video → events → score for all three games	12/12
souq_rules	correct, incorrect, mismatch, budget, duplicate, Arabic matching, tie-break	10/10

Speech check and what the tests found

A live call to the transcription model on synthetic text-to-speech audio returned: “I choose dates.” (1.75–2.55 s); “Coffee, please.” (5.85–6.75 s); “And the pottery.” (9.75–10.75 s). Deterministic scoring of that transcript gave **9** points (expected 9). Testing exposed three real defects that were fixed: the well’s own inlet was mis-counted as a leak; Arabic words with the ال prefix were not matched; and a host-accepted “possible goal” was not scored.

Limitations and honesty

- **No real family footage has been analysed yet.** Fixtures are synthetic renders with known ground truth; they test the pipelines, not real-world accuracy. No accuracy percentage is claimed. Real clips are to be recorded and reported separately.
- Football assumes a fixed camera, single-colour ball, contrasting shirts and calibrated goals; colour segmentation is lighting-sensitive; shirt numbers are not read.
- Falaj/Souq read printed markers; a hidden or damaged marker counts as absent. Speech timing is approximate (≈ ±1 s).
- Detection scores are heuristic indicators, not calibrated probabilities.

6 Fairness, privacy, deployment, impact and conclusions

Fairness and transparency

- **Rules are written, versioned and deterministic.** The AI proposes events; the rule engine decides points, so the same evidence always gives the same score.
- **Uncertainty is visible.** Unobserved or ambiguous moments become NEEDS_REVIEW and score nothing until the host accepts them; unobserved ball time is shown as “unknown possession”, not guessed.
- **Every call is contestable:** the host can accept, reject or clear any event; the score is recomputed and the decision is logged with name and time in a hash-chained audit trail.
- **Inclusive by design:** team and individual formats, Arabic/English interface and speech, heritage themes (falaj, souq) and equipment families can print at home.

Privacy and security

- Video is processed only to produce the judging report; uploads are limited in size and stored on the judge server, not shared with third parties other than the transcription call (audio only) when a key is configured.
- No faces are identified or stored as biometric data: players are colour tracks mapped to names the host types. Families should record only their own game and obtain consent from everyone filmed.
- All credentials stay server-side in environment variables (.env is git-ignored); the browser never sees them. Reports are signed to detect tampering.

Deployment

The repository **wasl-arena** holds frontend, backend, tests, tools, documentation and AI-usage records. The React app deploys from GitHub to Vercel (`frontend/vercel.json`, `VITE_API_URL`); the FastAPI judge deploys to Render (`render.yaml`) because video processing needs a long-running Python runtime with FFmpeg and OpenCV. Production build verified locally (TypeScript clean, Vite build). Live URLs are recorded in the README once the accounts are connected; publication is not claimed here until verified.

Impact and future work

WASL Arena shows that AI can make family gaming **more interactive** (real physical play), **more inclusive** (Arabic and English, team and solo), **culturally meaningful** (falaj and souq) and **transparently fair** (replayable evidence). Next steps: record and evaluate real family footage per game; add pretrained detectors and jersey-number reading to reduce calibration needs; add whistle/“goal!” audio cues to Football; and let hosts define their own games from the same rule-engine template.

Conclusion

A fair referee is not one that is always right: it is one that **shows its evidence, admits doubt and lets people decide**. WASL Arena delivers that for three games, with a working prototype, a verified pipeline and honest limits.

References

[1] Abu Dhabi University College of Engineering, “Family AI Game Day: Competition Questions – Cycle 3 Track (2026)”. [2] OpenCV ArUco marker detection documentation. [3] FFmpeg documentation. [4] FastAPI, React, Vite and Three.js documentation. [5] Google Gemini API, audio understanding. [6] RFC 2104, HMAC: Keyed-Hashing for Message Authentication. [7] UAE falaj irrigation systems, UNESCO World Heritage listing (Al Ain cultural sites).